

Nome: Yvens Ian Prado Porto

RA: 184031

Ferramentas e bibliotecas utilizadas: Python 3, Scapy (sniffing e inspeção profunda de pacotes), Pandas (para manipulação e agregação dos logs), Matplotlib e Seaborn (para visualização de dados), e ambiente de containers Docker.

Arquivos alterados: roteador.py (implementação da lógica do IPS e registro de logs) e network_analyse.ipynb (script de análise de dados).

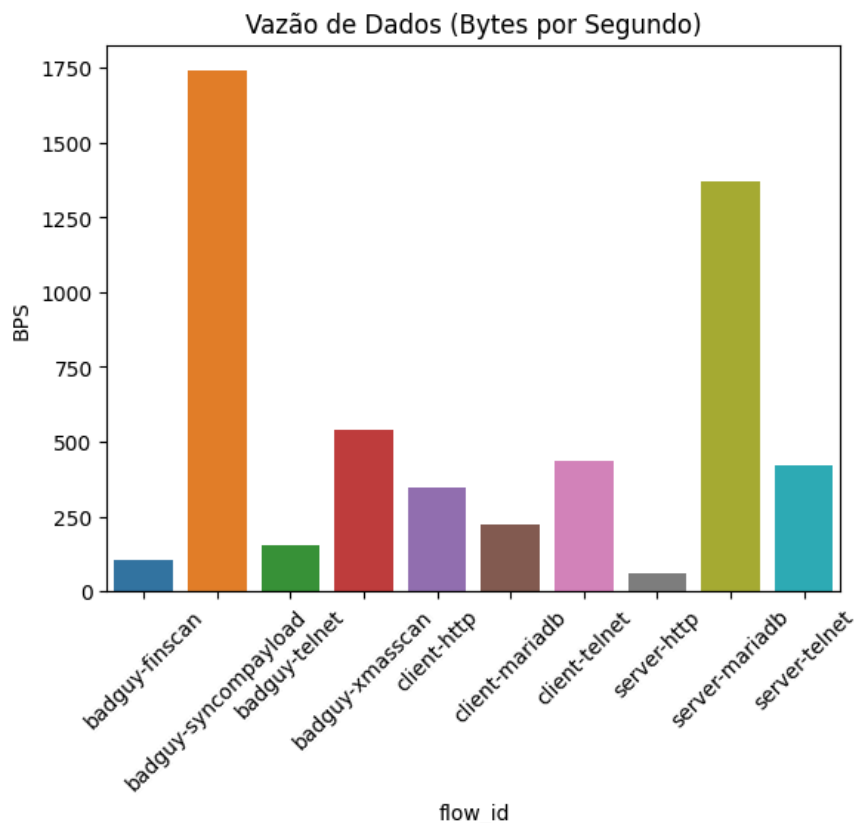
Análise Estatística do Tráfego (Sniffer)

O sniffer foi construído utilizando a biblioteca Scapy em Python, atuando como um roteador e firewall (IPS) entre duas redes distintas. A função central utilizada foi a `sniff()`, configurada para escutar simultaneamente as interfaces de ambas as redes. Para garantir que o roteador não perdesse pacotes ou sofresse vazamento de memória (*Out of Memory*) sob alta vazão, o parâmetro `store=0` foi estritamente utilizado. Dessa forma, a função `sniff()` não armazena os pacotes na RAM, repassando-os imediatamente para a função de callback definida no parâmetro `prn` (neste caso, a função `forward_packet`), onde o pacote é analisado, registrado em um arquivo CSV e, se for legítimo, roteado através da função `sendp()`.

Filtros e Captura

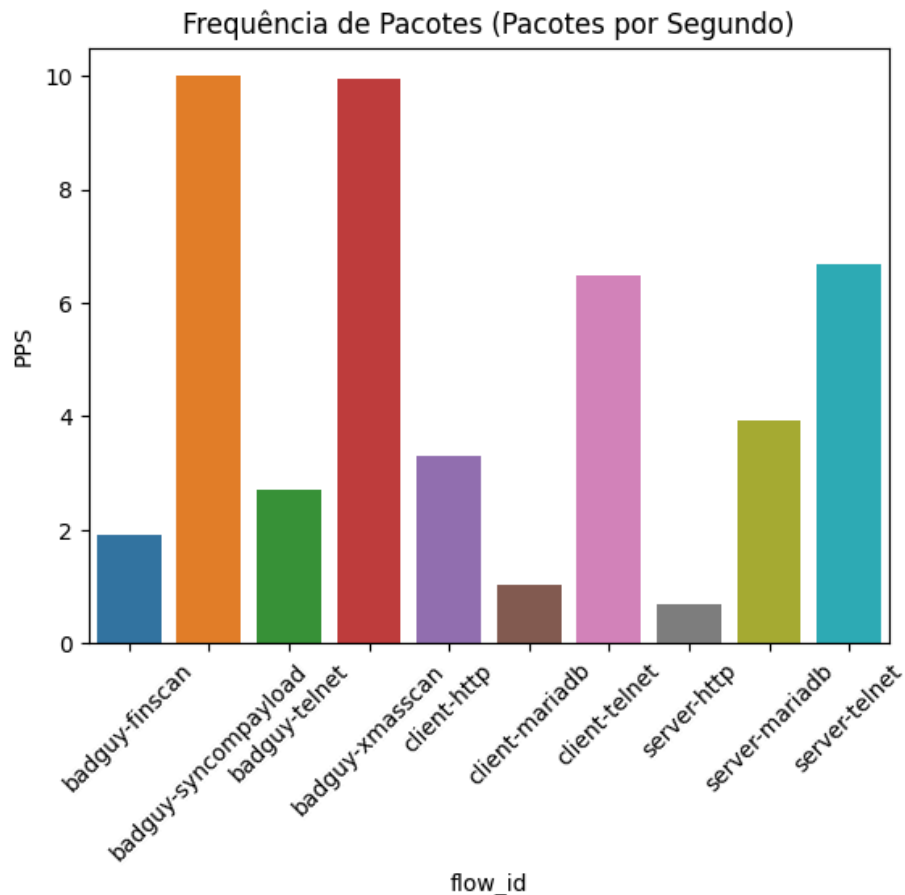
- **Filtros BPF (Berkeley Packet Filter) utilizados / Tratamento em Nível de Aplicação:** Embora a captura tenha sido feita de forma ampla nas interfaces, a filtragem estrutural ocorreu logo na entrada do callback, utilizando métodos do próprio Scapy como `pkt.haslayer(IP)` e `pkt.haslayer(TCP)`.
- **Justificativa:** Essa abordagem de filtro foi escolhida para descartar rapidamente tráfegos de broadcast local ou protocolos não roteáveis (como ARP e STP) que não faziam parte do escopo de análise, focando o processamento da CPU apenas na inspeção das flags TCP e no registro do tráfego das camadas 3 e 4.

Vazão por Serviço (BPS)



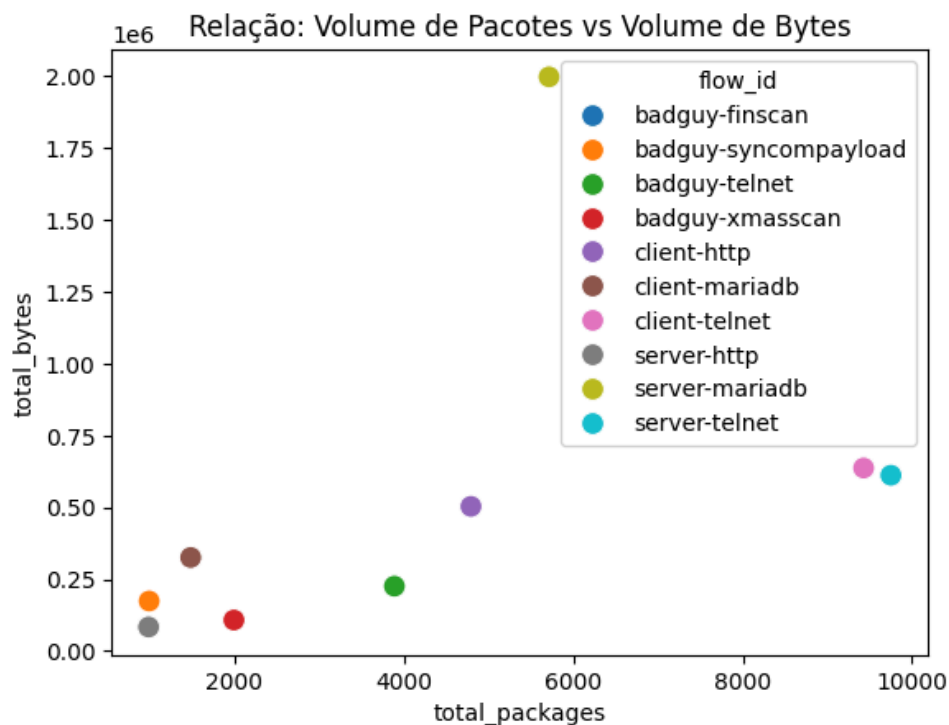
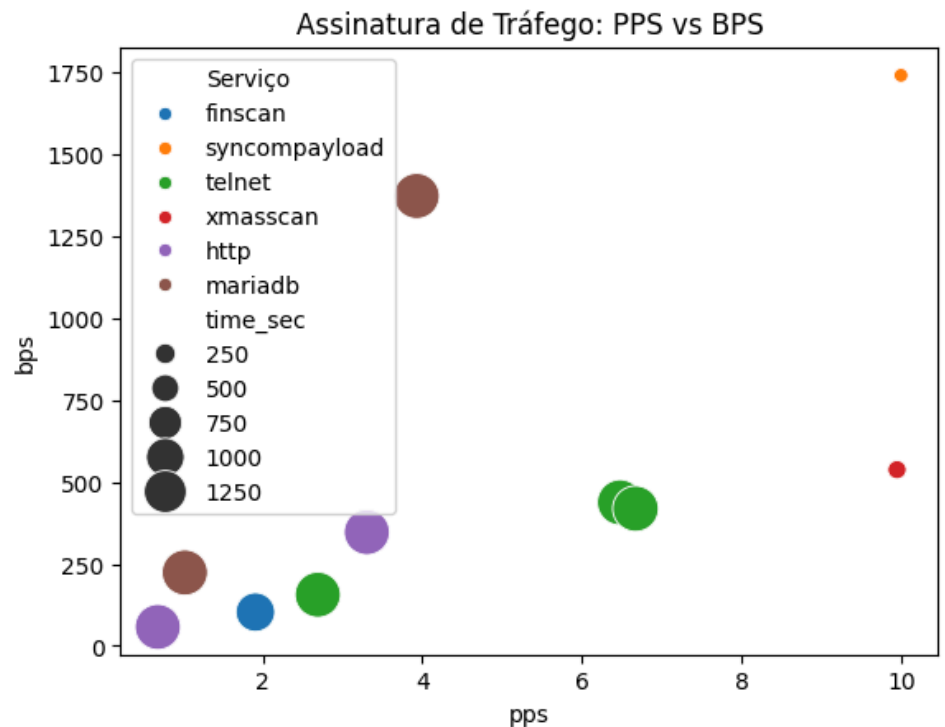
- **Análise Descritiva:** O gráfico de vazão (Bytes por Segundo) demonstra o impacto volumétrico de diferentes tipos de tráfego. Espera-se que o tráfego legítimo (como as respostas do server-mariadb) possua as maiores vazões devido à transmissão de dados reais. No entanto, o destaque do gráfico é o pico anômalo do fluxo badguy-syncompayload. Ao cruzar esse dado com a configuração do ataque, constata-se que a ferramenta Hping3 foi configurada para injetar 120 bytes de carga útil (*payload* de lixo) em 1000 pacotes enviados em alta velocidade (*--fast*). Esse comportamento caracteriza não apenas uma varredura, mas uma tentativa de ataque de Negação de Serviço (SYN Flood) focada em exaurir a largura de banda. O alto índice de BPS comprova a agressividade do ataque, que foi contido com sucesso pelo IPS.

Frequência de Pacotes (PPS)



- **Análise Descritiva:** A análise de PPS revela a verdadeira "assinatura" volumétrica dos fluxos na rede. Como esperado teoricamente, o tráfego de banco de dados (mariadb) possui alta vazão em bytes, mas trafega uma quantidade menor de pacotes por segundo (por volta de 4 PPS), pois envia blocos grandes de dados. Em contraste, os fluxos do atacante disparam drasticamente no gráfico de frequência: as varreduras de badguy-syncompayload e badguy-xmasscan atingem os picos absolutos da captura na tentativa de inundar o alvo e mapear portas rapidamente. Vale notar também o comportamento do tráfego legítimo de telnet, que apresenta um PPS moderadamente alto (~6.5 PPS); isso é característico do protocolo, que transmite interativamente pequenos pacotes a cada tecla digitada pelo usuário, mas ainda assim se mantém em um volume seguro e inferior ao gerado pelo atacante.

Volume de pacotes pelo volume de bytes



- Análise descritiva:** A análise do gráfico de dispersão que relaciona o volume total de pacotes com o volume de bytes permite mapear o comportamento absoluto dos fluxos da rede durante a captura. Observa-se que o tráfego legítimo domina os extremos do plano cartesiano: o servidor de banco de dados (server-mariadb) atinge o maior pico de volume de dados, o que reflete o envio de respostas pesadas contendo múltiplas consultas. Em contrapartida, as sessões de Telnet (tanto cliente quanto servidor) acumulam a maior quantidade absoluta de pacotes com um volume de bytes consideravelmente menor. Isso é esperado dada a natureza interativa do

protocolo, que trafega pequenos dados a cada interação do usuário no terminal. Os fluxos maliciosos do atacante concentram-se próximos à origem do gráfico, o que indica que as rajadas de ataque, apesar de intensas, foram bloqueadas ou tiveram curta duração, resultando em um volume total absoluto pequeno em comparação às sessões legítimas.

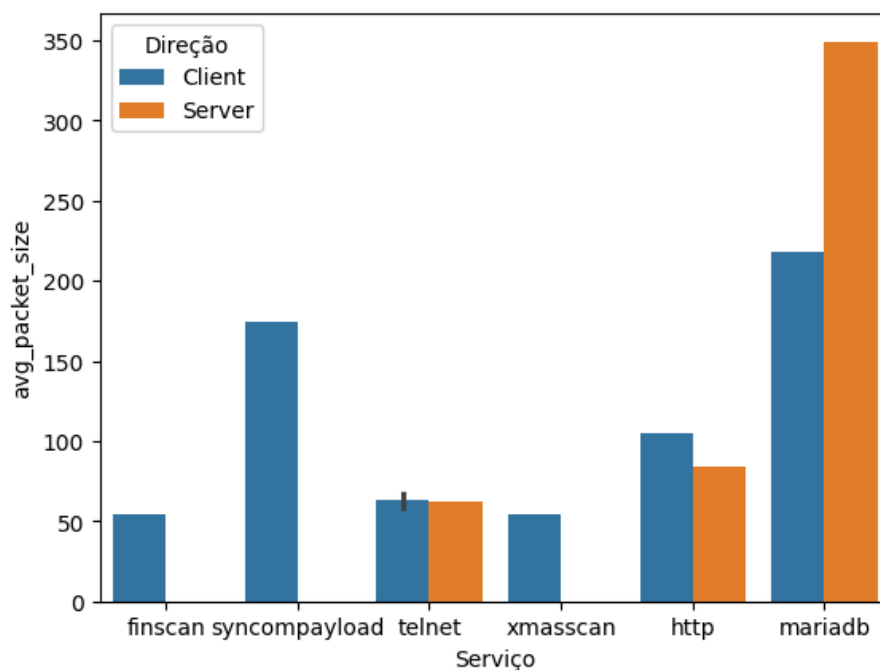
Quando essas grandezas são normalizadas pelo tempo no gráfico de Assinatura de Tráfego (PPS vs. BPS), a "impressão digital" de cada serviço torna-se evidente através de agrupamentos bem definidos. Os ataques de varredura, como o XMAS Scan, isolam-se em regiões de altíssima frequência de pacotes (PPS) e baixíssima vazão de dados (BPS), revelando o comportamento típico de envio em massa de cabeçalhos vazios para mapeamento de portas. Em contraste, a tentativa de ataque com o Hping3 (SYN com payload) destaca-se com as maiores taxas em ambos os eixos, caracterizando uma clara tentativa de exaustão volumétrica e consumo de banda (DoS). Por fim, o tráfego legítimo obedece aos seus padrões teóricos, com o banco de dados operando em alta vazão e baixa frequência de pacotes. Essa separação comprova que o sistema é capaz de perfilar e distinguir anomalias na rede baseando-se puramente na assinatura comportamental e geométrica do tráfego.

Detecção de Anomalias e Processamento de Firewall

Nesta etapa, o sniffer atua como um elemento de rede (roteador/firewall) que decide o destino do pacote com base no seu comportamento.

2.1 Comparativo Visual: Fluxo Normal vs. Anômalo

Apresenta a comparação lado a lado de como as métricas se comportam quando um ataque é injetado.



A comparação do tamanho médio do pacote revela a anatomia da anomalia. O tráfego normal apresenta tamanhos médios que variam de 70 bytes (Telnet) a mais de 200-300 bytes (MariaDB/HTTP). Por outro lado, os ataques injetados (nullscan, xmasscan, finscan) cravam uma média em torno de 54 bytes. A exceção é o syncompayload, que foi uma tentativa de evasão e injeção do atacante usando Hping3, mitigada com sucesso pelo nosso IPS.

2.2 Método de Detecção Implementado

O critério de detecção foi estritamente focado em Assinaturas de Flags TCP (Inspeção Profunda de Pacotes), ignorando completamente endereços IP e portas, o que evitou que técnicas de evasão como o IP Spoofing (onde o atacante forja seu IP de origem) tivessem sucesso.

Fluxo de Execução:

- O pacote é capturado pela interface física e passa pela verificação `haslayer(TCP)`.
- O método `is_malicious()` extrai as flags TCP do pacote.
- Ocorre a comparação heurística:
 1. Se as flags contiverem FPU simultaneamente, a assinatura acusa XMAS Scan.
 2. Se as flags estiverem ausentes (0), a assinatura acusa NULL Scan.
 3. Se a flag conter apenas F, a assinatura acusa FIN Scan.
 4. Se a flag for S (SYN) mas houver carga útil na camada Raw, classifica-se como injeção/DoS.
- Encaminhamento / Drop: Caso o método retorne falso para anomalias, o pacote é validado como "PERMITIDO", o MAC Address de destino é resolvido (com cache) e a função `sendp()` roteia o pacote. Caso o método retorne verdadeiro para anomalia, um evento "BLOQUEADO" com a razão específica é gravado no log e o pacote é silenciosamente descartado (*Drop*), sem notificar o invasor.

2.3 Eficácia do Firewall

- **Resultado:** O método foi altamente eficaz, bloqueando as tentativas de varredura (Scans) independentemente dos IPs falsos gerados pelo atacante.
- **Descoberta e Atualização (Patching):** Durante a primeira fase laboratorial, a análise de dados revelou um volume anômalo classificado genericamente como "Telnet" originado pelo atacante. Uma auditoria cruzada revelou tratar-se de um Falso Negativo: um ataque de FIN Scan gerado pela ferramenta hping3. Como o IPS validava assinaturas estáticas, o tráfego não catalogado logrou êxito inicial. A eficácia foi atestada ao incluirmos a regra de detecção do FIN Scan no código, o que erradicou a evasão, conforme provam os gráficos finais.
- **Conclusão (Latência e Segurança):** O processamento de um "roteador virtual" com IPS em camada de aplicação Python (Scapy) impacta diretamente a latência da rede (aumentando o *Round-Trip Time*), pois obriga o pacote a transitar do *Kernel Space* para o *User Space* a fim de ter seus cabeçalhos TCP desmontados e avaliados antes do repasse. No entanto, o custo-benefício computacional se justifica pelo drástico aumento de segurança para o host final, blindando o servidor contra

sondagens maliciosas e esgotamento de recursos causados por ataques automatizados. É importante ressaltar que ataques como o SYN Scan puro ainda atravessam regras *stateless* sem payload, evidenciando a necessidade futura de firewalls *Stateful* para proteção holística.

Declaração de Uso de Inteligência Artificial

Neste laboratório, a inteligência artificial (Google Gemini) foi utilizada como uma ferramenta de apoio para tirar dúvidas pontuais de programação e ajudar na análise dos resultados.

A IA foi utilizada especificamente nas seguintes partes:

- **Na manipulação de dados (Pandas):** Para entender e corrigir um erro de "KeyError" que ocorreu na hora de agrupar a tabela final no Jupyter Notebook.
- **Na geração dos gráficos:** Para montar os comandos básicos das bibliotecas Matplotlib e Seaborn usados para desenhar os gráficos do relatório.
- **No debate teórico:** Para discutir o comportamento de uma anomalia nos gráficos gerados, o que me auxiliou a entender que o script do atacante estava realizando um *FIN Scan* e um *SYN Flood* que passavam direto pelas regras iniciais do firewall.

Toda a montagem da topologia no Docker, a captura dos pacotes via Scapy, a escrita da lógica de bloqueio no roteador.py e a interpretação final do trabalho foram desenvolvidas por mim.